

Module 1

Manipulation de fichiers automatique

os · pathlib · shutil

Basé sur un apprentissage réel avec les datasets Data-foundations-month1

1. Ce que ces outils font vraiment

Quand tes fichiers sont déjà bien organisés (comme dans datasets/), tu n'as besoin que de pathlib pour les pointer et les lire avec pandas.

Quand tu reçois des fichiers en désordre (comme en entreprise), les 3 bibliothèques entrent en jeu ensemble :

Bibliothèque	Rôle concret
pathlib	Pointe vers les fichiers, lit les noms et extensions, inspecte les tailles
shutil	Déplace physiquement les fichiers vers les bons dossiers
os	Renomme les fichiers sur le disque

Remarque : La logique : pathlib inspecte shutil trie os renomme pandas analyse

2. Erreurs fréquentes sur Windows

Erreur backslash — SyntaxError

```
# Mauvais
dossier = Path("C:\Users\HP\Documents\Data")

# Correct — raw string avec r""
dossier = Path(r"C:\Users\HP\Documents\Data")
```

Remarque : Le r devant les guillemets dit à Python de ne pas interpréter les \ comme des caractères d'échappement.

Erreur nom de variable — NameError

```
# Mauvais — datasets n'est pas défini
datasets.exists()

# Correct — utilise le nom que tu as défini
dossier_absolu = Path(r"C:\Users\HP\...\datasets")
dossier_absolu.exists() # True
```

Remarque : pathlib ne connaît que les noms de variables que tu lui donnes, pas les noms de dossiers réels.

3. pathlib — avec tes fichiers réels

Pointer vers ton dossier

```
from pathlib import Path

datasets = Path(r"C:\Users\HP\Documents\Projets de
compétences\Data\Data-foundations-month1\datasets")

print(datasets.exists()) # True
```

Lister tous tes CSV

```
for fichier in datasets.glob("*.csv"):
    print(fichier.name)
```

Résultat obtenu :

```
coffee_sales.csv
coffee_sales_clean.csv
comptage-voyageurs-trains-transilien.csv
tmdb_clean.csv
TMDB_Data.csv
transilien_clean.csv
```

Séparer fichiers bruts et fichiers nettoyés

```
print("--- Fichiers bruts ---")
for fichier in datasets.glob("*.csv"):
    if "_clean" not in fichier.name:
        print(fichier.name)

print("--- Fichiers nettoyés ---")
for fichier in datasets.glob("*.csv"):
    if "_clean" in fichier.name:
        print(fichier.name)
```

Inventaire complet de tes datasets

```
import pandas as pd
from pathlib import Path

datasets = Path(r"C:\Users\HP\...\datasets")
print("=== INVENTAIRE DES DATASETS - MOIS 1 ===\n")

for fichier in datasets.glob("*.csv"):
    df = pd.read_csv(fichier)
    taille_ko = round(fichier.stat().st_size / 1024, 1)
    print(f"Fichier      : {fichier.name}")
```

```

print(f"Lignes      : {len(df)}")
print(f"Colonnes    : {len(df.columns)}")
print(f"Taille      : {taille_ko} Ko")
print("-" * 40)

```

4. Simulation entreprise — dossier test_entreprise

Pour pratiquer les cas réels, on crée un dossier avec des fichiers en désordre :

```

test_entreprise/
├── Ventes Avril 2026.csv
├── Ventes Fevrier 2026.csv
├── VENTES-MARS-2026.csv
├── Rapport Client.xlsx
├── budget_2026.xlsx
├── données_brutes.json
├── notes reunion.txt
└── fichier_vide.csv      <- vide exprès

```

5. Étape 1 — Trier les fichiers par extension (shutil)

```

from pathlib import Path
import shutil

source = Path(r"C:\Users\HP\Downloads\test_entreprise")
destination_base = Path(r"C:\Users\HP\Downloads\test_entreprise")

categories = {
    ".csv": "csv",
    ".xlsx": "excel",
    ".json": "json",
    ".txt": "texte"
}

for fichier in source.iterdir():
    if fichier.is_file():
        extension = fichier.suffix.lower()
        if extension in categories:
            dossier_cible = destination_base / categories[extension]
            dossier_cible.mkdir(parents=True, exist_ok=True)
            shutil.move(fichier, dossier_cible / fichier.name)
            print(f"Déplacé : {fichier.name} -> {categories[extension]}/")
        else:
            print(f"Extension non reconnue : {fichier.name}")

```

Résultat obtenu :

```

Déplacé : budget_2026.xlsx -> excel/
Déplacé : données_brutes.json -> json/
Déplacé : fichier_vide.csv -> csv/
Déplacé : notes reunion.txt -> texte/
Déplacé : Rapport Client.xlsx -> excel/
Déplacé : ventes_avril_2026.csv -> csv/
Déplacé : Ventes Fevrier 2026.csv -> csv/
Déplacé : VENTES-MARS-2026.csv -> csv/

```

6. Étape 2 — Renommer en snake_case (os)

```
from pathlib import Path
import os

dossiers = [
    Path(r"C:\Users\HP\Downloads\test_entreprise\csv"),
    Path(r"C:\Users\HP\Downloads\test_entreprise\excel"),
    Path(r"C:\Users\HP\Downloads\test_entreprise\json"),
    Path(r"C:\Users\HP\Downloads\test_entreprise\texte")
]

for dossier in dossiers:
    for fichier in dossier.iterdir():
        if fichier.is_file():
            nouveau_nom = fichier.stem.lower()
            nouveau_nom = nouveau_nom.replace(" ", "_")
            nouveau_nom = nouveau_nom.replace("-", "_")
            nouveau_nom = f"{nouveau_nom}{fichier.suffix.lower()}"
            nouveau_chemin = fichier.parent / nouveau_nom
            if fichier.name != nouveau_nom:
                os.rename(fichier, nouveau_chemin)
                print(f"Renommé : {fichier.name} -> {nouveau_nom}")
            else:
                print(f"Inchangé : {fichier.name}")
```

Résultat obtenu :

```
Inchangé : fichier_vide.csv
Renommé : ventes avril 2026.csv -> ventes_avril_2026.csv
Renommé : Ventes Fevrier 2026.csv -> ventes_fevrier_2026.csv
Renommé : VENTES-MARS-2026.csv -> ventes_mars_2026.csv
Inchangé : budget_2026.xlsx
Renommé : Rapport Client.xlsx -> rapport_client.xlsx
Inchangé : données_brutes.json
Renommé : notes reunion.txt -> notes_reunion.txt
```

Comprendre : nouveau_chemin = fichier.parent / nouveau_nom

Partie	Valeur exemple
fichier.parent	C:\...\test_entreprise\csv
nouveau_nom	ventes_fevrier_2026.csv
fichier.parent / nouveau_nom	C:\...\csv\ventes_fevrier_2026.csv

Remarque : os.rename() a besoin du chemin complet des deux côtés, pas juste le nom du fichier.

7. Étape 3 — Valider les fichiers vides (pathlib)

```
from pathlib import Path
import pandas as pd

dossier_csv = Path(r"C:\Users\HP\Downloads\test_entreprise\csv")

for fichier in dossier_csv.glob("*.csv"):
    taille = fichier.stat().st_size
    if taille == 0:
        print(f"ATTENTION fichier vide : {fichier.name}")
        print("-" * 40)
    else:
        df = pd.read_csv(fichier)
        print(f"OK : {fichier.name} - {len(df)} lignes")
        print("-" * 40)
```

Résultat obtenu :

```
ATTENTION fichier vide : fichier_vide.csv
-----
OK : ventes_avril_2026.csv - 0 lignes
OK : ventes_fevrier_2026.csv - 0 lignes
OK : ventes_mars_2026.csv - 0 lignes
```

Remarque : Sans cette vérification, pandas essaie de lire un fichier vide et fait planter tout le pipeline, y compris les fichiers valides qui suivent.

8. À retenir

Opération	Code
Pointer vers un dossier	<code>Path(r"C:\ton\chemin")</code>
Vérifier qu'il existe	<code>dossier.exists()</code>
Lister les CSV	<code>dossier.glob("*.csv")</code>
Lire nom / extension	<code>fichier.name</code> / <code>fichier.suffix</code>
Taille en octets	<code>fichier.stat().st_size</code>
Déplacer un fichier	<code>shutil.move(source, destination)</code>
Renommer un fichier	<code>os.rename(ancien_chemin, nouveau_chemin)</code>
Construire un chemin	<code>dossier / "sous_dossier" / "fichier.csv"</code>

9. Réflexion — Framework personnel

Ce que j'ai fait

...

Pourquoi

...

Ce que ça m'a appris

...

Module suivant → Module 2 : Traitements pandas automatisés